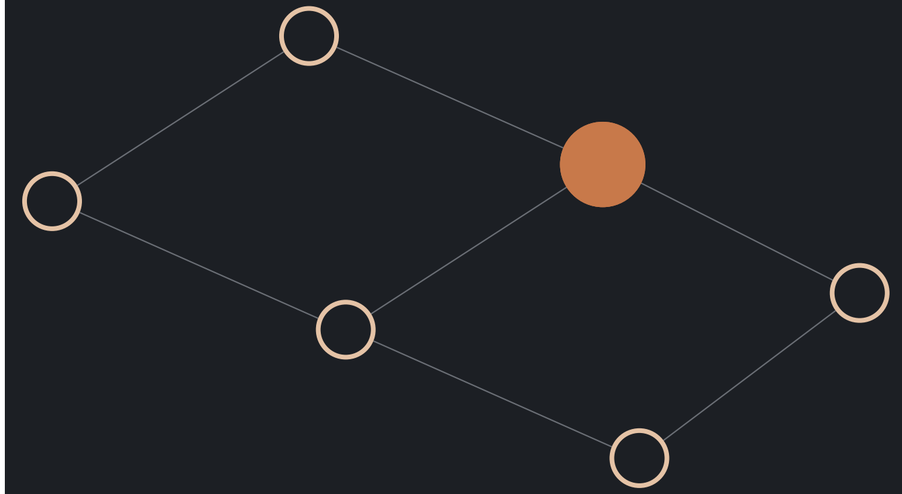
DuocUC® | ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

PROYECTO DE TÍTULO · PTY4614

TimeFlow

Monitoreo del trabajo remoto mediante un canvas de nodos



Alberto Asenjo · Andre Muñoz · Yhoventino Albornoz

Ingeniería en Informática · Escuela de Informática y Telecomunicaciones

01

Problemática y contexto

Cliente real: un proyecto audiovisual de música infantil que funciona desde 2021

Sus colaboradores trabajan desde distintas ubicaciones y en distintos horarios. Para el administrador, el trabajo a distancia era difícil de medir y de respaldar.

LA RESPUESTA

TimeFlow registra el tiempo y el avance de cada tarea en un canvas de nodos conectados, con evidencia y supervisión para el administrador.

1

Sin registro de horas

No había forma de saber cuánto tiempo dedicaba cada colaborador ni a qué tareas.

2

Sin evidencia del avance

El trabajo a distancia se reducía a mensajes y archivos sueltos.

3

Herramientas sin relaciones

Trello o Monday ordenan tareas en listas y tableros, pero no muestran cómo una tarea se conecta con otra.

02

Objetivos

Los seis objetivos específicos están implementados y en uso

OBJETIVO GENERAL

Diseñar e implementar una plataforma web escalable para el monitoreo colaborativo del proyecto audiovisual, con registro de tiempo y tareas en un canvas de nodos e informes en PDF para el administrador.

01 Calendario mensual con botones de ingreso y salida por día.

02 Contabilizar horas por fecha y generar un conteo semanal.

03 Registro diario de tareas desde un listado predefinido.

04 Árbol de expansión: una tarea deriva en un nuevo proceso con sus propias horas.

05 Interfaz como grafo de nodos conectados con el detalle de cada etapa.

06 Módulo de supervisión con un resumen del trabajo para el administrador.

03

Metodología: Scrum

Sprints de dos semanas, con revisión del incremento junto al cliente al cierre de cada sprint

1

Sprint Planning

Se priorizan historias de usuario del backlog.

2

Desarrollo + Daily

Sincronización diaria y tablero Scrum.

3

Sprint Review

El cliente prueba el incremento y da feedback.

4

Retrospectiva

El equipo define mejoras para el siguiente sprint.

Roles del equipo

Scrum Master / QA: **Alberto Asenjo (Líder de proyecto)**
Product Owner: Andre Muñoz (Business Data Analyst)
Desarrollo: Yhoventino Albornoz (Backend y Frontend)

Seguimiento: backlog en 7 épicas con historias de usuario y criterios de aceptación, y GitHub para versiones y evidencias.

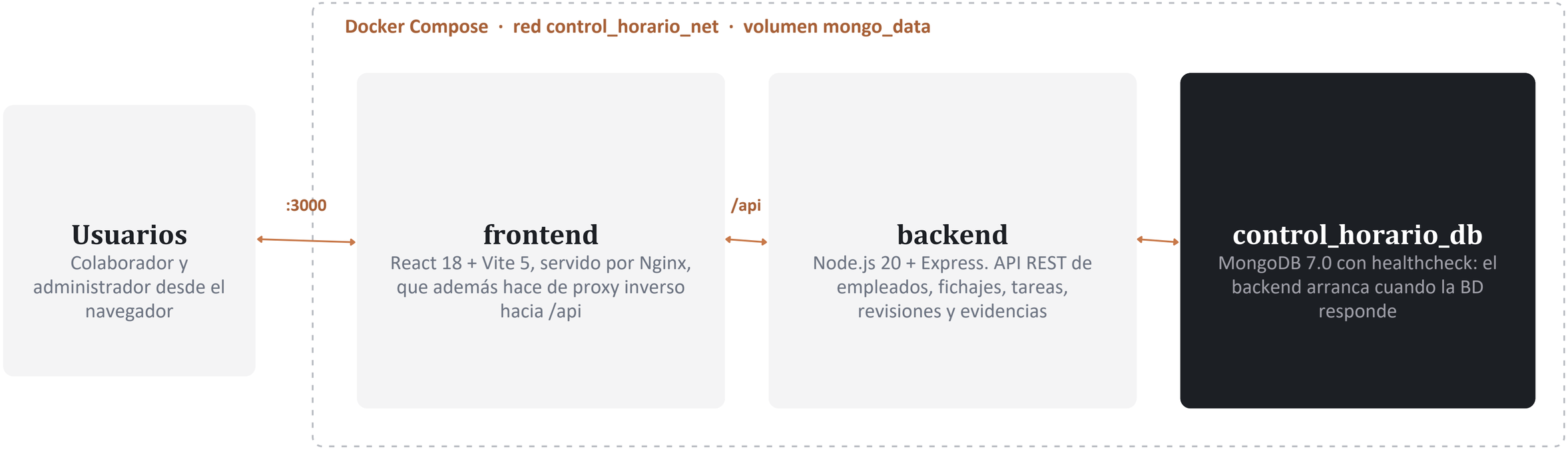
¿Por qué Scrum?

- El alcance cambió: del registro por calendario a un canvas de nodos. Scrum permitió repriorizar sin rehacer la planificación.
- Cada sprint entregó un incremento que el cliente pudo validar.
- La interfaz de nodos se prototipó temprano, no al final como en cascada.

04

Arquitectura de la solución

Aplicación web de tres capas, cada una en su propio contenedor



¿Por qué esta arquitectura?

Separación de responsabilidades
Interfaz, API y datos se actualizan por separado.

Un solo lenguaje
JavaScript en frontend y backend; JSON en la base de datos.

Entorno reproducible
docker compose up levanta los 3 servicios igual en cualquier equipo.

05

Componentes y tecnologías

Versiones tomadas del código del proyecto (package.json, Dockerfile y docker-compose)

Frontend

React 18.3 · Vite 5.4

- lucide-react (íconos) y clsx
- Canvas de nodos, Gantt y gráficos hechos a medida, sin librerías externas
- MediaRecorder API para notas de voz

Por qué
Componentes reutilizables y compilación rápida.

Backend / API

Node.js 20 · Express 4.19

- Mongoose 8.5 como ODM
- cors, morgan (logs) y dotenv
- nodemon en desarrollo

Por qué
Mismo lenguaje que el frontend; API REST simple.

Base de datos

MongoDB 7.0

- Esquemas con validación (required, enum, unique)
- Volumen persistente mongo_data
- Cálculo de horas en el modelo

Por qué
Documentos flexibles: el registro diario guarda tareas y evidencias.

Infraestructura

Docker · Nginx · GitHub

- Docker Compose: 3 servicios y healthcheck
- Build en 2 etapas: Node 20 → Nginx Alpine
- Nginx: servidor SPA y proxy /api

Por qué
Mismo entorno en todos los equipos.

06

Modelo de datos

MongoDB 7.0 con Mongoose: 3 colecciones; tareas, pausas y evidencias embebidas en el registro diario

employees

```
_id, code (único)
firstName, lastName
email (único)
department, position
pin
role: EMPLOYEE | ADMIN
avatar, active
```

timerecords

```
_id
employee → employees
date (YYYY-MM-DD)
clockIn, clockOut
status: WORKING | ON_BREAK
        | COMPLETED
totalWorkMinutes
totalBreakMinutes
breaks [ ] · tasks [ ]
taskBreakdown, areaBreakdown
```

tasks [ ] (embebido)

```
taskName, area, priority
start, end, durationMinutes
status: IN_PROGRESS |
 UNDER_REVIEW | APPROVED
 | REJECTED
reviewedBy → employees
captures[ ], voiceNotes[ ]
isTransversal
```

globaltasks

```
_id, name (único)
category, description
active
```

breaks [ ] (embebido)

```
start, end, reason
durationMinutes
```

Decisión de diseño: un documento por colaborador y día. La jornada se lee en una sola consulta y calculateMinutes() obtiene las horas y el desglose por tarea y área.

07

Seguridad

Riesgos identificados, qué está resuelto y qué falta

RIESGO	MEDIDA	ESTADO
Acceso según perfil	Login con código o email + PIN; roles EMPLOYEE y ADMIN dirigen a vistas distintas	Implementado
Borrado accidental de datos	El endpoint /api/seed está bloqueado en producción (403)	Implementado
Datos inválidos	Validación en esquemas Mongoose y manejo global de errores	Implementado
Credenciales en el código	Configuración por variables de entorno (.env + Docker Compose)	Implementado
PIN guardado en texto plano	Cifrar contraseñas con bcrypt	Pendiente
API sin token por ruta	JWT con expiración + middleware que valide el rol en cada endpoint	Pendiente
Superficie expuesta	Restringir CORS (hoy '*'), no publicar el puerto 27017 y usar HTTPS	Pendiente

07

Seguridad

Riesgos identificados, qué está resuelto y qué falta

RIESGO	MEDIDA	ESTADO
Acceso según perfil	Login con código o email + PIN; roles EMPLOYEE y ADMIN dirigen a vistas distintas	Implementado
Borrado accidental de datos	El endpoint /api/seed está bloqueado en producción (403)	Implementado
Datos inválidos	Validación en esquemas Mongoose y manejo global de errores	Implementado
Credenciales en el código	Configuración por variables de entorno (.env + Docker Compose)	Implementado
PIN guardado en texto plano	Cifrar con bcrypt	Pendiente
API sin token por ruta	JWT con expiración + middleware que valide el rol en cada endpoint	Pendiente
Superficie expuesta	Restringir CORS (hoy '*'), no publicar el puerto 27017 y usar HTTPS	Pendiente

08

Nivel de madurez

Estado real de la solución al momento de la entrega

EN USO POR EL CLIENTE

El proyecto audiovisual ya utiliza TimeFlow en su trabajo diario.

Implementado

- Fichaje: iniciar, pausa con motivo, reanudar y finalizar, con cronómetro en vivo
- Horas de hoy, la semana y el mes, con desglose por tarea y área
- Canvas de nodos: proyectos y subtareas que se mueven, conectan y desconectan
- Evidencias por tarea: capturas, notas de voz y conocimiento transversal
- Revisión del administrador: aprobar o rechazar tareas con comentarios
- Monitor en vivo, gestión de empleados, catálogo de tareas, Gantt, exportación CSV e informes PDF
- Chat del equipo y perfil editable con foto

Limitaciones

- Solo versión web, sin app móvil
- Autenticación aún sin tokens ni cifrado
- No probado con equipos grandes

Próximos pasos

- Seguridad: bcrypt, JWT, CORS y HTTPS
- Despliegue en la nube
- Pruebas automatizadas
- Aplicación móvil híbrida

09

Modelo de negocio

1

Propuesta de valor

- Muestra el avance del trabajo remoto como un grafo, no como una lista
- Registro de horas y evidencia sin vigilar la pantalla
- Informes PDF listos para el administrador

2

Usuarios y clientes

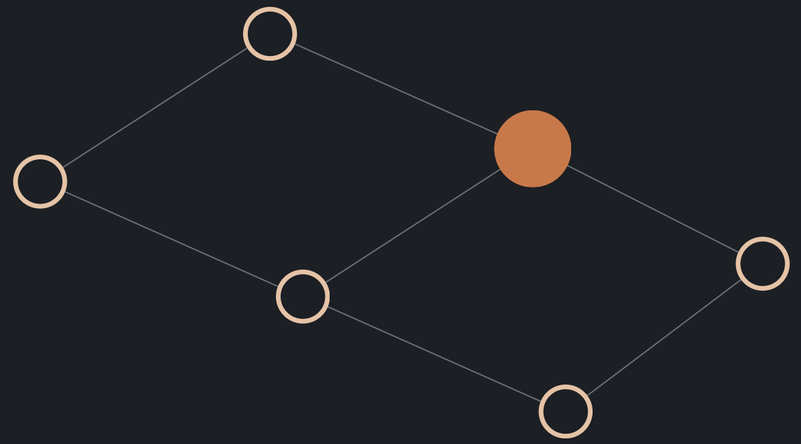
- Equipos creativos pequeños: audiovisual, diseño, animación
- Estudios y freelancers que trabajan a distancia
- Primer cliente: el proyecto audiovisual, que ya lo usa a diario

3

Implementación y escala

- Cliente actual como caso de éxito
- Servicio web por suscripción mensual por usuario
- Contenedores Docker listos para la nube

“No competimos mostrando más filas por pantalla. Competimos mostrando cómo se mueve el trabajo.”

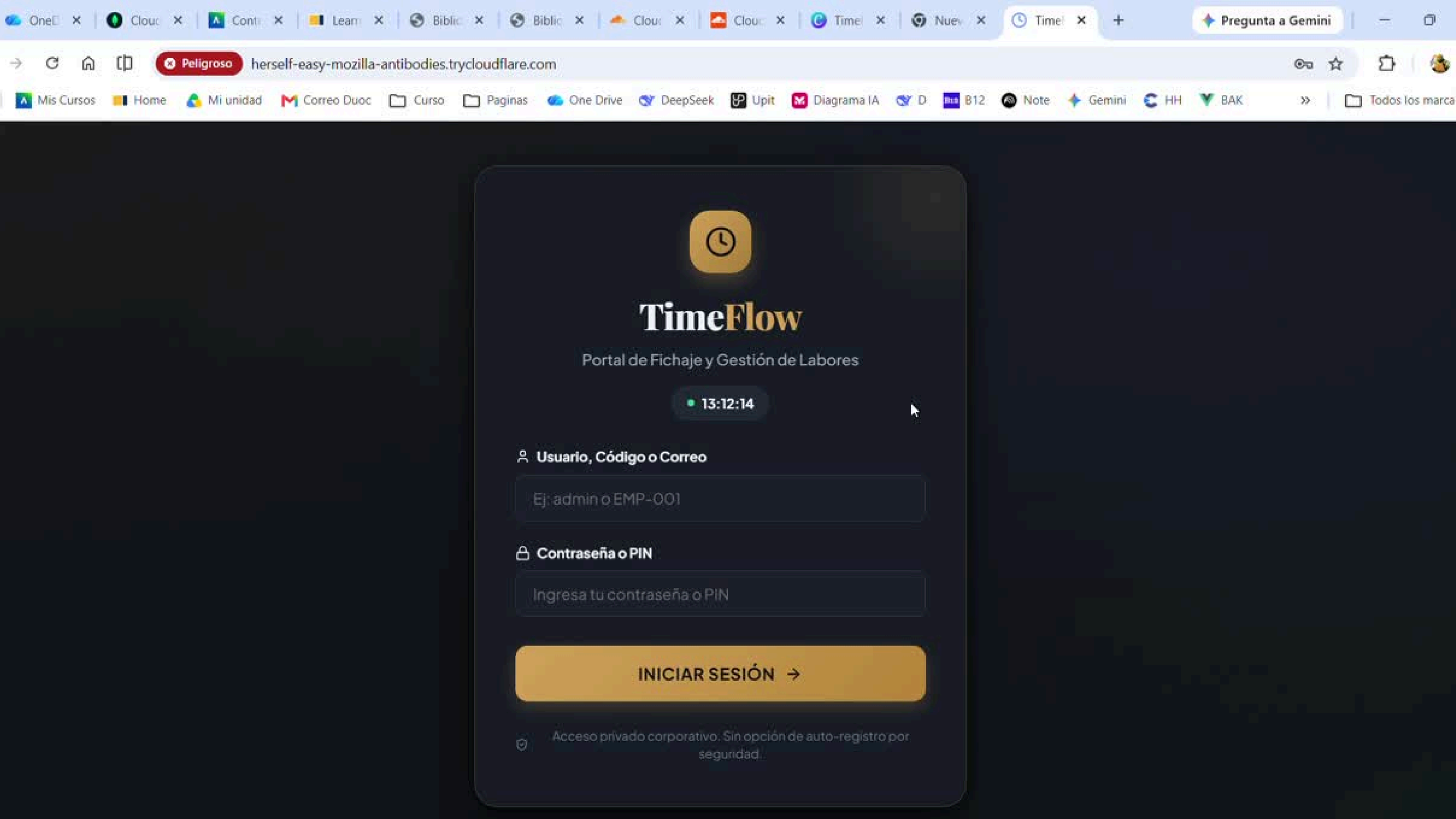


DEMOSTRACIÓN

TimeFlow en uso

Video de 2 minutos

Login → Proyecto → Subtareas → Cronómetro → Captura → Chat → Informe PDF


Pregunta a Gemini
Peligroso
herself-easy-mozilla-antibodies.trycloudflare.com
Mis Cursos
Home
Mi unidad
Correo Duoc
Curso
Paginas
One Drive
DeepSeek
Upit
Diagrama IA
D
B12
Note
Gemini
HH
BAK
Todos los marca
TimeFlow
Portal de Fichaje y Gestión de Labores
13:12:14
Usuario, Código o Correo
Ej: admin o EMP-001
Contraseña o PIN
Ingresa tu contraseña o PIN
INICIAR SESIÓN →
Acceso privado corporativo. Sin opción de auto-registro por seguridad.

10 Conclusión

01 TimeFlow resuelve la falta de registro y seguimiento del trabajo de un equipo remoto, y hoy ya lo utiliza el cliente en su trabajo diario.

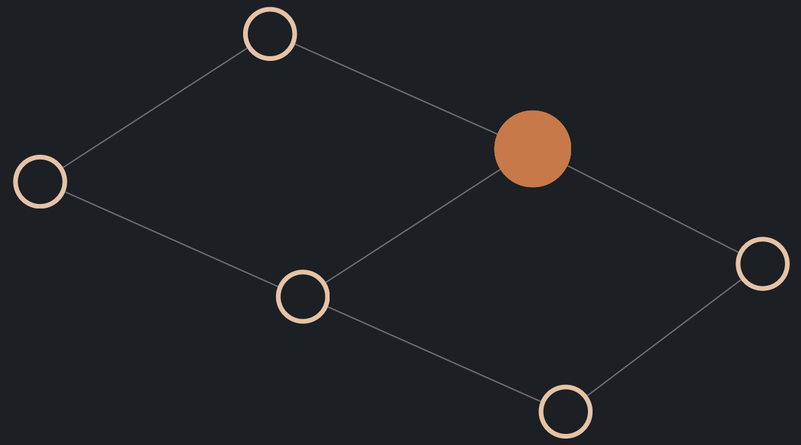
02 El canvas de nodos muestra cómo se conectan las tareas, y el cronómetro, las capturas y los informes PDF dan evidencia del avance.

03 Scrum permitió absorber el cambio de alcance y validar cada incremento con el cliente.

04 La arquitectura en contenedores deja la solución lista para crecer; el siguiente paso es reforzar la seguridad y desplegarla en la nube.

COMISIÓN

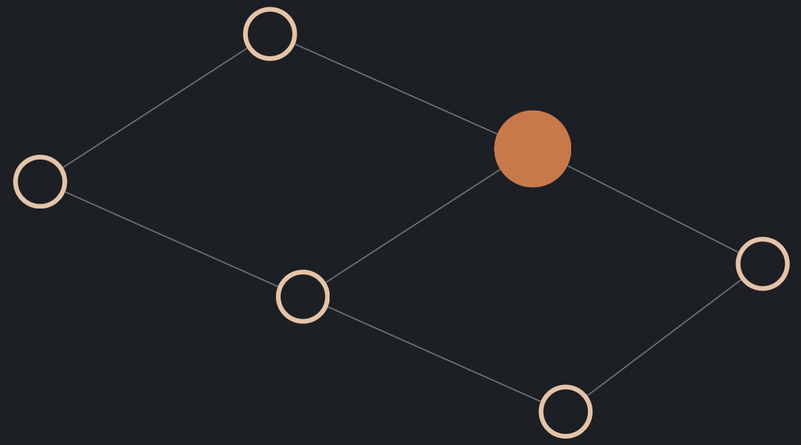
Preguntas



Duoc UC | ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

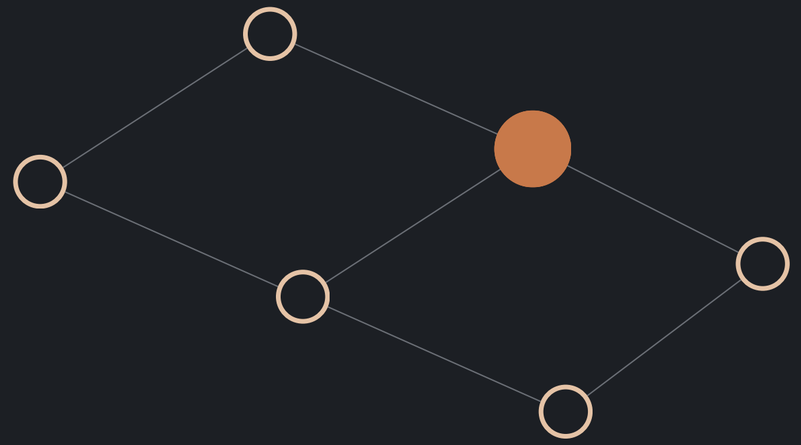
Muchas gracias

Alberto Asenjo · Andre Muñoz · Yhoventino Albornoz



MATERIAL DE RESPALDO

Anexos

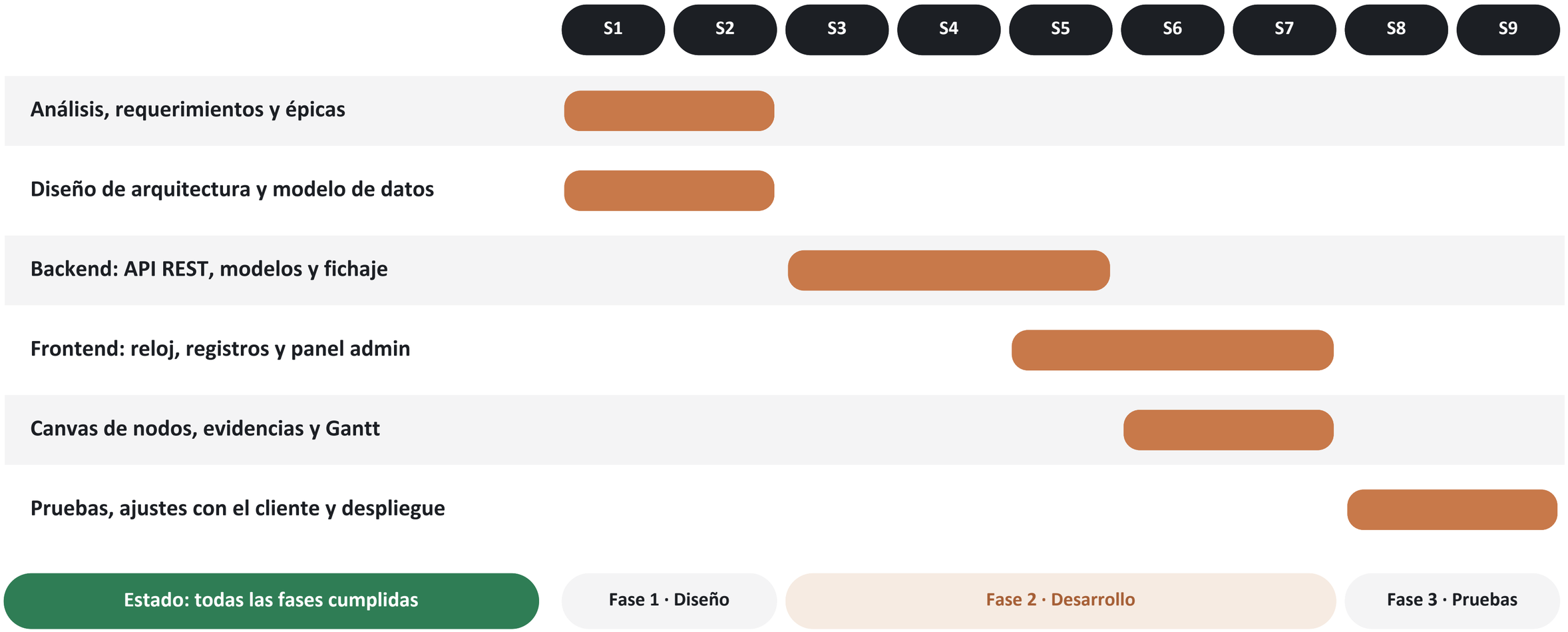


A · Cronograma B · Obstáculos C · API REST D · Flujo de una jornada E · Datos personales F · Épicas

A

Cronograma del proyecto

9 sprints de 2 semanas (18 semanas). Avances respaldados en GitHub en las revisiones de las semanas 6, 11 y 17



B

Obstáculos y aprendizajes

Cambio de alcance

Problema: El cliente pidió pasar de un registro por calendario a un canvas de nodos.
Solución: Se repriorizó el backlog por épicas y el canvas se prototipó temprano.

Visualizaciones sin librerías

Problema: El canvas, el Gantt y los gráficos no calzaban con librerías estándar.
Solución: Se construyeron como componentes propios en React.

Arranque de los contenedores

Problema: El backend fallaba si MongoDB aún no estaba lista.
Solución: Healthcheck en Docker Compose y reintentos de conexión.

Datos reales en producción

Problema: Riesgo de borrar datos del cliente al reiniciar la base de demo.
Solución: /api/seed responde 403 cuando NODE_ENV es production.

Aprendizaje: con un cliente real hay que priorizar, comunicar los cambios y proteger sus datos desde el primer día.

C

API REST

Endpoints expuestos por el backend Express

POST	`/api/employees/login`	Inicio de sesión con código o email + PIN
GET · POST	`/api/employees`	Listar y crear colaboradores
GET · PUT · DELETE	`/api/employees/:id`	Ver, editar y eliminar un colaborador
GET · POST · DELETE	`/api/tasks`	Catálogo global de tareas
POST	`/api/records/punch`	Fichaje: CLOCK_IN, START/SWITCH/COMPLETE_TASK, BREAK_START/END, CLOCK_OUT
GET	`/api/records/active`	Colaboradores trabajando en este momento
GET	`/api/records/summary/:employeeId`	Horas del colaborador por período
GET	`/api/records/stats/admin`	Indicadores del panel de administración
GET · POST	`/api/records/pending-reviews` · `/tasks/approve-all`	Tareas por revisar y aprobación masiva
POST	`/api/records/:recordId/tasks/:taskId/review` · `/evidence`	Aprobar o rechazar una tarea; adjuntar evidencia
GET	`/api/health`	Estado del servicio

D

Flujo de una jornada

De la entrada del colaborador a la revisión del administrador

1 Login

Código o email + PIN. El rol define la vista.

`POST /employees/login`

2 Inicio de jornada

Se crea el registro del día (timerecord).

`punch: CLOCK_IN`

3 Tareas

Inicia, cambia o completa tareas con área y prioridad.

`START / SWITCH / COMPLETE`

4 Pausas y evidencia

Pausa con motivo; capturas y notas de voz por tarea.

`BREAK_* · /evidence`

5 Fin de jornada

calculateMinutes() suma horas y desglose por tarea y área.

`punch: CLOCK_OUT`

6 Revisión admin

Aprueba o rechaza cada tarea con comentarios.

`/review · approve-all`

E

Datos personales

Qué datos trata TimeFlow y qué marco legal aplica

Datos que trata TimeFlow

- Identificación: nombre, código, email, cargo y área
- Credencial de acceso (PIN) y foto de perfil
- Horarios de entrada, salida y pausas, con su motivo
- Dirección IP del fichaje
- Capturas de pantalla y notas de voz como evidencia

Se usan solo para medir horas y avance del trabajo.

Ley N.º 19.628

Protección de la vida privada, vigente. Regula el tratamiento de datos personales en Chile.

Ley N.º 21.719

Nueva ley de protección de datos personales. Entra en vigencia el 1 de diciembre de 2026 y crea la Agencia de Protección de Datos Personales.

Cómo nos preparamos

- Cifrar el PIN y proteger la API con tokens (pendiente)
- Informar al colaborador qué datos se registran y para qué
- Las evidencias las sube el colaborador: no hay vigilancia en segundo plano

F

Épicas del backlog

Cada épica se divide en historias de usuario con criterios de aceptación

EPIC-01 Autenticación, seguridad y control de acceso (RBAC)

EPIC-02 Registro de jornada, reloj digital y fichaje en vivo

EPIC-03 Medición y gestión de labores y áreas en tiempo real

EPIC-04 Bitácora personal y registro histórico del colaborador

EPIC-05 Panel de supervisión, auditoría y reportes para administradores

EPIC-06 Gestión integral de la plantilla de colaboradores

EPIC-07 Arquitectura multi-servicio, contenedorización y UI/UX

Priorizadas por el Product Owner en cada Sprint Planning.